Kimi on llama.cpp: memory bandwidth, power and energy

Kimi-Linear-48B-A3B (Q8_0 GGUF, 52.25 GB) on the DGX Spark (GB10) · llama.cpp · 22 runs, 2026-10-12 to 10-14 · sources: docs/10-kimi-llamacpp-moe-sweep.md, runs/kimi-q8-llamacpp/results.md, progress.md

Key takeaways

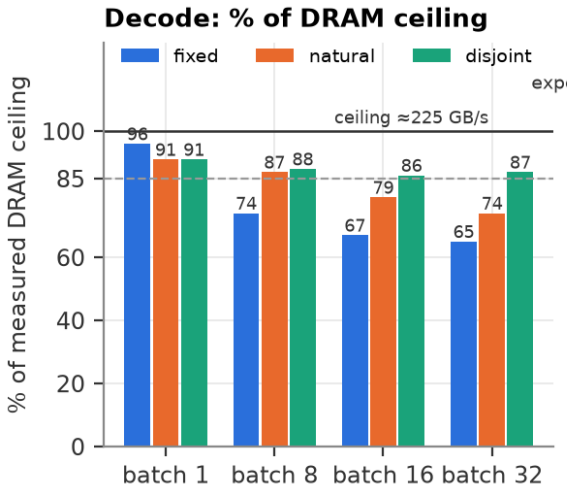
- **Batch-1 decode runs at the memory limit.** Each step reads 3.60 GB at 205–214 GB/s, 91–96% of the measured ~225 GB/s ceiling. The floor is 16.0 ms per step; measured is 16.9–17.6 ms. Only fewer bytes can make it faster.
- **Batching moves the limit.** Disjoint routing stays at 86–88% of the ceiling. Fixed routing falls to 65% at batch 32, where per-sequence KDA (linear-attention) state work, not weights, takes the time.
- **The GPU is almost never idle.** In unprofiled runs idle time is ~0 at batch 1 and at most ~6 ms per step (4–8%) at batch 32; the 4–17% idle seen under nsys is mostly the profiler's own overhead.
- **Power is nearly constant (26–31 W in decode), so energy follows bytes per token.** Batch 1 costs ~0.50 J per token in every routing mode, ~17.5 pJ of GPU energy per DRAM bit.
- **Batching saves energy only when tokens share experts:** at batch 32, 8× less per token for fixed, 3.7× for natural, 1.8× for disjoint (no gain beyond batch 8).
- Energy is the **GPU rail only**: GB10 does not report LPDDR5X or CPU power. Read the figures as lower bounds, ±10–15%.

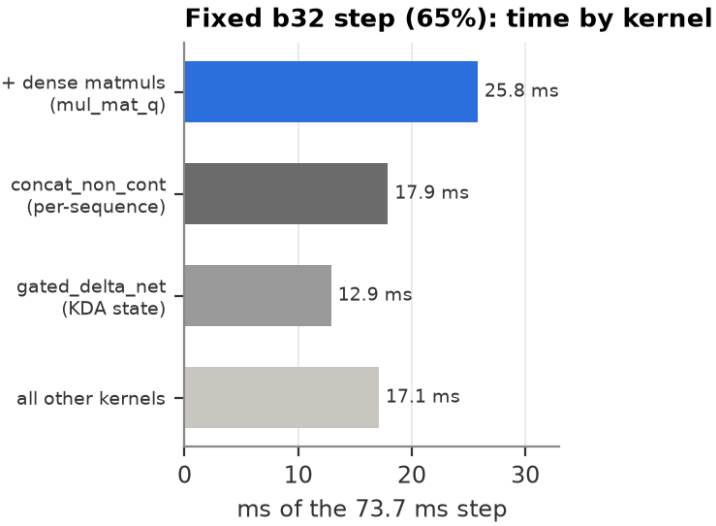
What was measured, and how

	Bandwidth	Power and energy
Source	Exact DRAM bytes per kernel from ncu tier 1 (application replay), summed per decode step or prefill	NVML cumulative GPU energy counter, plus 20 ms power.draw samples as a cross-check
Time base	The unprofiled baseline run (ncu's own replayed durations are not performance)	A separate unprofiled pass per run: each phase repeated until the window is ≥ 10 s
Reference	Ceiling = streaming-kernel bandwidth measured just before each run (223–227 GB/s)	Idle = 8 s with the model resident and nothing running (9.4–10.6 W)
Derived	GB/s, % of ceiling, L2 hit rate, L2 bytes per DRAM byte	W, J per token, J above idle, pJ per DRAM bit (all GPU energy ÷ DRAM bits moved)
Accuracy	Bytes exact; batch-1 decode reads 1.05× the analytic prediction	Counter updates every ~100 ms; cross-checks 0.84–1.08; ±10–15% absolute, tighter between runs

Runs. Kimi has 256 experts in each MoE layer and each token uses 8. Three routing modes bracket what a step must read: **fixed** (every token uses experts 0–7, the floor), **natural** (the model's own router) and **disjoint** (no two tokens share an expert, the ceiling). Decode: batch 1, 8, 16 and 32 per mode (128-token prompt, 32 new tokens). Prefill: 16, 64 and 512-token prompts per mode, plus one 16,384-token natural run. The 128-token prefill comes from the batch-1 decode runs.

Decode bandwidth: at the ceiling until the batch grows

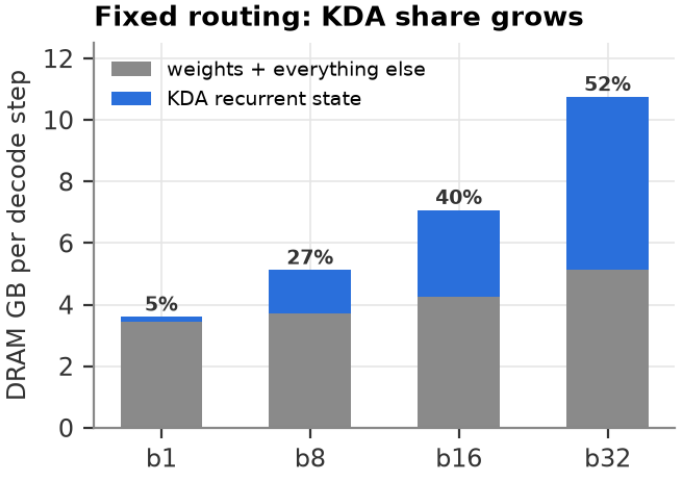




Above ~85% of the ceiling a run is limited by DRAM; below it, time goes to kernels that move few bytes. At batch 1 every mode reads the same 8 experts per layer and runs at 91–96%. Disjoint keeps streaming new weights as the batch grows (each row adds 8 experts, ~1.9 GB per token), so it stays at 86–88%. Natural (79% and 74% at b16 and b32) and fixed (down to 65%) share experts, so they read fewer weight bytes and per-sequence work shows. **Right:** in fixed b32, concat_non_cont takes 17.9 ms but moves almost no DRAM (latency-bound, likely the KDA conv-state concatenation), and gated_delta_net another 12.9 ms.

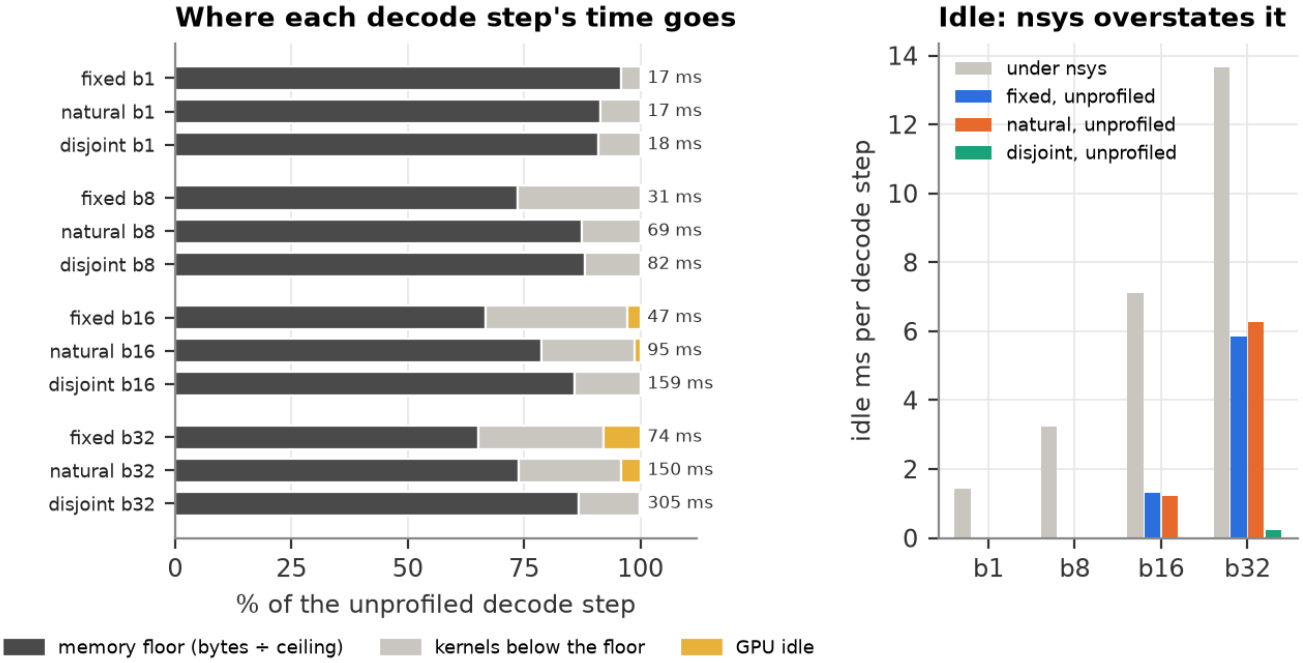
Where decode bytes go

Batch-1 decode step	GB
Weights: 8 experts/layer + dense + lm_head	3.35
KDA state, moved ~4× (budget 0.042)	0.17
MLA latent cache	0.05
Other activations	0.03
Measured total (predicted 3.44)	**3.60**



A batch-1 step reads 3.60 GB against a predicted 3.44 GB (1.05×). Nearly all of the +0.16 GB is the KDA (linear-attention) recurrent state: k_get_rows_float_vec gathers it and gated_delta_net_cuda reads it and writes the new one, 0.17 GB against the 0.042 GB budgeted, about 4% of batch-1 decode time. The state is 0.175 GB per sequence, so in fixed routing it reaches 5.6 GB at batch 32: 52% of all decode traffic, more than all the weights read. **L2 hit rate** in batch-1 decode is 13% (1.15 L2 bytes per DRAM byte): the weights stream through once with no reuse, as expected.

Decode time: memory floor, slower kernels, and idle

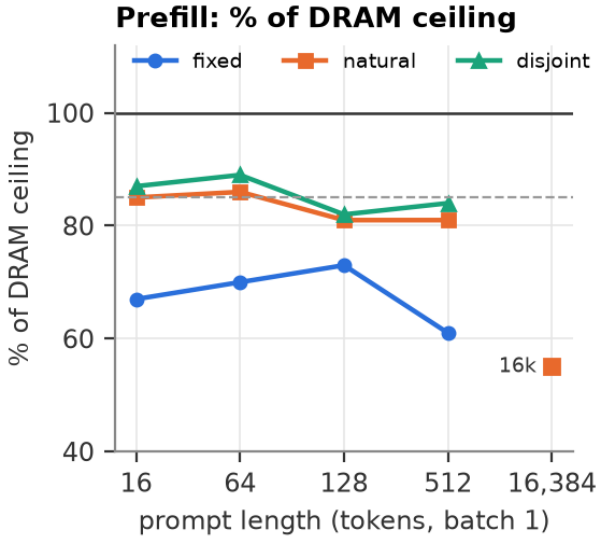


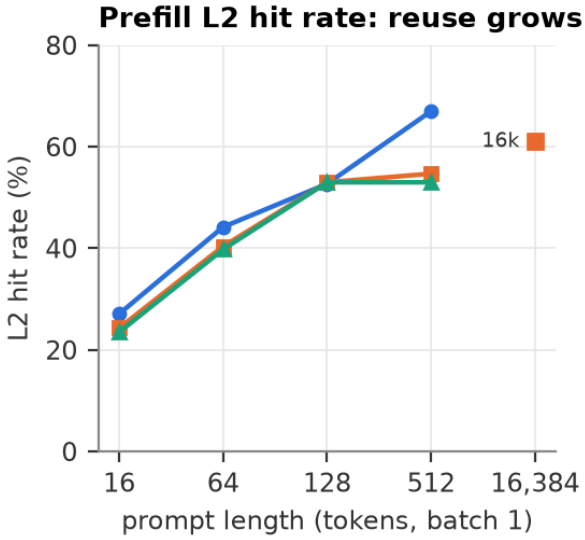
Each unprofiled decode step is split into three parts. **Memory floor** = the step's exact DRAM bytes ÷ the measured ceiling: the time it would take at full streaming speed. **GPU idle** = the step time minus the time the GPU spends executing kernels (kernel time from nsys). **Kernels below the floor** = the rest: the GPU is busy, but not moving data at full speed (small or latency-bound kernels).

- **Batch 1: 91–96% floor, no idle.** The 0.7–1.5 ms above the floor is kernels that run below full bandwidth, not gaps. llama.cpp keeps the GPU fully fed.
- **Disjoint at any batch: 86–91% floor, idle ≤ 0.2 ms.** Streaming new experts dominates.
- **Fixed and natural at batch 32: real idle appears**, ~6 ms per step (8% of fixed's 73.7 ms, 4% of natural's 150 ms), likely host-side per-sequence work. Fixed b32 spends 27% of its step below the floor (concat_non_cont, gated_delta_net; page 2) and 8% idle.
- **Short fixed prefills** (16–128 tokens) are the other place with real idle: 3–4 ms of a 29–52 ms prefill (7–13%), a fixed host cost that a few-expert prefill cannot hide.

Right: why this differs from the busy % reported so far. Under nsys every decode step shows 1.4 ms of idle at batch 1, rising to ~13.5 ms at batch 32, the same in every routing mode. But nsys's own kernel time per step already matches the unprofiled step time to within ±1 ms in most runs: the idle gaps are mostly added by nsys's tracing, not present in normal running. The "Busy" column in results.md (83–96% in decode) therefore understates how busy the GPU is; use this split instead.

Prefill bandwidth: still memory-bound up to 512 tokens





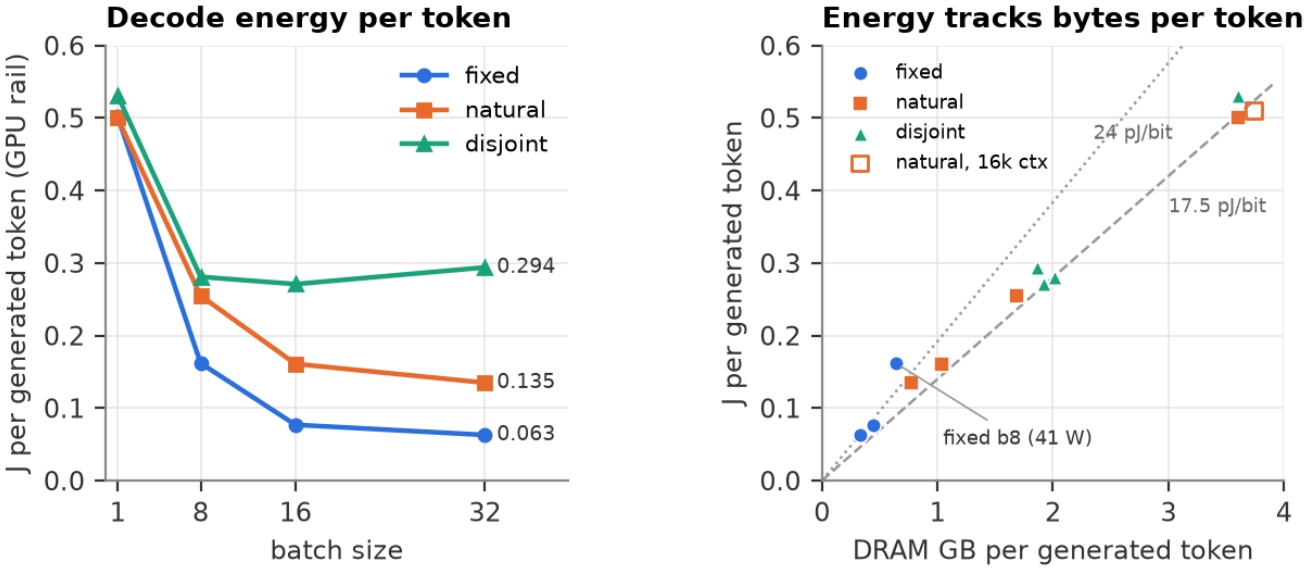
Natural and disjoint prefill run at 81–89% of the ceiling at every length up to 512 tokens. With 256 experts the tokens spread thin: at 512 tokens natural uses ~225 experts per layer, about 18 tokens each. Each expert's weights are reused by only a few tokens, so prefill behaves like a large decode. **Fixed is different:** all tokens hit the same 8 experts, its L2 hit rate climbs to 67% at 512 tokens, and it uses only 61–73% of the ceiling. It reads few weight bytes (4.3–21.9 GB) and moves towards compute: the remaining bytes are activations and attention, 36–61 MB per prompt token. **16k tokens:** 1.11 TB in 8.86 s, only 55% of the ceiling with 61% L2 hits. Attention and KDA work dominate there, not weight streaming.

Prefill DRAM bytes and bandwidth at a glance

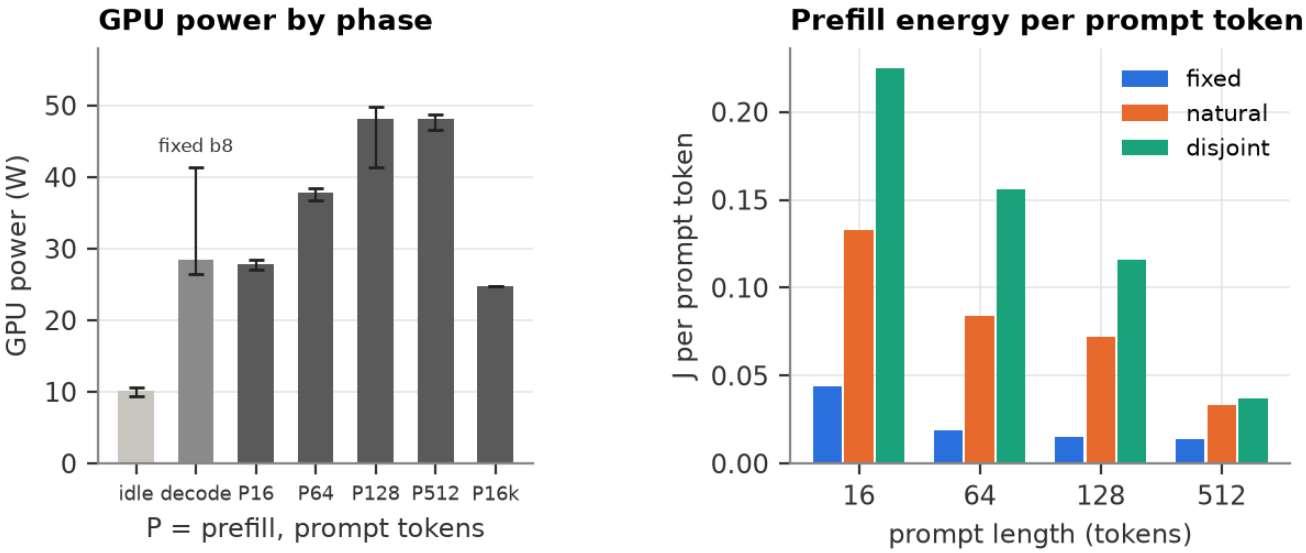
Prompt (tokens)	16	64	128	512	16,384
DRAM GB: fixed / natural / disjoint	4.3 / 15.0 / 27.0	6.1 / 28.1 / 53.4	8.5 / 36.7 / 56.6	21.9 / 64.6 / 74.4	– / 1,110 / –
% of ceiling: fixed / natural / disjoint	67 / 85 / 87	70 / 86 / 89	73 / 81 / 82	61 / 81 / 84	– / 55 / –
Prefill ms: fixed / natural / disjoint	29 / 79 / 138	38 / 145 / 269	52 / 202 / 305	159 / 353 / 399	– / 8,860 / –

All weights are 51.8 GB. From 512 tokens natural prefill reads more than the whole model, because the non-weight traffic (~18.5 GB at 512 tokens, measured with fixed routing) grows with the prompt.

Energy: set by the bytes each token needs



Left: batch 1 costs 0.50 / 0.50 / 0.53 J per token (fixed / natural / disjoint): same bytes, same energy. At batch 32: fixed 0.063 J (8× less), natural 0.135 J (3.7×), disjoint 0.294 J (1.8×, and worse than at batch 8). The 16k-context run costs 0.51 J, the same as a short context. **Right:** all 13 decode runs lie near one line of 17.5–24 pJ of GPU energy per DRAM bit. **Why:** power is almost constant, so energy = power × time, and time = bytes ÷ bandwidth. For natural b1: 28.4 W × (3.60 GB ÷ 205.8 GB/s) = 0.497 J against 0.500 J measured. Runs above the 17.5 line (fixed and natural at b16–b32) spend time in the low-traffic per-sequence kernels from page 2 while still drawing power.



Left (bar = typical, line = range over runs): the GPU is far from its power limit. Decode draws 26–31 W, about 3× idle; only fixed b8 is higher at 41 W (see caveats). Prefill draws more as the prompt grows (27 → 49 W) because larger matmuls keep more of the GPU busy. The 16k prefill draws only 24.7 W, lower than any other, which fits it being neither bandwidth- nor compute-saturated (55% of the ceiling). **Right:** prefill energy per token follows the experts touched (fixed < natural < disjoint, 3–5× apart at short prompts) and falls with length as weight reads are shared. At 512 tokens natural and disjoint converge (0.033 vs 0.037 J) because natural already touches most experts. 16k tokens: 498 J, 0.030 J per token. **Idle share:** about a third of batch-1 decode energy (0.17 of 0.50 J) is the ~10 W idle floor; batching spreads it over more tokens.

Decode energy per run

Batch	Mode	W	J/token	Above idle J/token	pJ/DRAM bit	GB/token	% of BW ceiling
1	fixed	29.5	0.502	0.328	17.5	3.60	96%
1	natural	28.4	0.500	0.334	17.4	3.60	91%
1	disjoint	29.8	0.531	0.347	18.5	3.60	91%
8	fixed	41.4	0.162	0.123	31.7	0.64	74%
8	natural	29.4	0.255	0.167	18.9	1.69	87%
8	disjoint	27.5	0.281	0.178	17.5	2.01	88%
16	fixed	26.4	0.077	0.048	21.9	0.44	67%
16	natural	27.4	0.161	0.101	19.3	1.04	79%
16	disjoint	27.1	0.271	0.169	17.6	1.92	86%
32	fixed	27.5	0.063	0.040	23.6	0.34	65%
32	natural	29.6	0.135	0.088	21.7	0.78	74%
32	disjoint	30.8	0.294	0.194	19.8	1.86	87%
1 (16k ctx)	natural	27.1	0.510	0.314	17.0	3.74	90%

The six moe-prefill-* workloads are left out: their decode energy is not valid (caveat 2).

Caveats

1. GPU rail only. NVML on GB10 reports GPU-memory and module power as N/A, so LPDDR5X, CPU and the rest of the module are missing. True system energy per token is higher, most of all for the bandwidth-bound runs. pJ/bit is all GPU energy per DRAM bit, not the energy of the DRAM itself.

2. Decode energy of the six moe-prefill-* workloads is not valid. They generate 4 tokens, so each decode segment is 3 steps (~55 ms), shorter than the counter's ~100 ms update. Their cross-checks are 0.82–1.68. Their prefill energy is valid.

3. Accuracy. NVIDIA publishes no accuracy for GB10's power telemetry. Valid cross-checks range 0.84–1.08 (the sampled-power check reads 0.84–0.95 on most decode runs; the continuous-window check stays at 0.99–1.08). Quote absolute values as ±10–15%; comparisons between runs are tighter.

4. Fixed b8 is an outlier at 41 W and 32 pJ/bit (every other decode run: 26–31 W). Its cross-checks are clean (1.04, 0.99). It has the highest decode L2 hit rate (52%), i.e. more on-chip work per DRAM byte, but the cause is not confirmed.

5. The 16k run's low prefill power (24.7 W) is not explained yet: low-occupancy attention/KDA kernels, or the SW Power Cap that ncu flagged during that run's replay.

6. Idle time is inferred, not measured directly: unprofiled step time minus nsys kernel time, so nsys's small effect on kernel durations sets its accuracy (about ±1 ms per step; a few runs come out slightly negative, shown as 0). The 16k prefill is left out: under nsys it took 12.6 s against 8.9 s unprofiled.

7. Prompts are synthetic random token IDs. Real text may route differently, which moves natural's bytes and energy.

Next steps

- Reduce KDA state traffic (gather + read + write, ~4× the minimum) and the per-sequence concat_non_cont time: the main limit for batched Kimi on llama.cpp, in both bandwidth and energy.
- Re-measure decode energy for the short workloads with a longer generation; repeat fixed b8 to confirm the 41 W.
- Add a wall-plug or module-level power reading to get system energy, including LPDDR5X.